Today's agenda

- Introduction to Heap/PQ. → PriorityQueue
- K smallest element. +1
- median of an array. → {leetcode Hard}

AlgoPrep

// Introduction

	insert (n)	getmin()	deletemin()
Arraylist	O(1)	O(N)	O(N)
LinkedList	O(N)/O(1)	O(N)	O(N)
Queue	O(1)	O(N)	O(N)
Hashmap	O(1)	O(N)	O(N)
PQ	O(logN)	O(1)	O(logN)

Smart Array

Min PQ

```
PriorityQueue<Integer> PQ = new PriorityQueue<>();
```

name

Max PQ

```
PriorityQueue<Integer> PQ = new PriorityQueue<>();
```

Collections.reverseOrder()

PQ

no. of elements in PQ

```
→ PQ.add(10);
→ PQ.add(20);
→ PQ.add(15);
→ PQ.add(5);
→ PQ.remove();  → 5
→ PQ.peek();    → 10
→ PQ.size();    → 3
```

↳ O(1)

↳ PQ.add(n) → add value n → O(logN)

↳ PQ.remove() → remove min ele → O(logN)

↳ PQ.peek() → get the min ele.

AlgoPrep

Q) Kth Smallest element

↳ Given N distinct elements, Point K smallest elements.

Ex: arr[10] = { 8 3 10 4 11 2 7 6 14 1 }
(indexes: 0 1 2 3 4 5 6 7 8 9)

K=4: 1 2 3 4

arr[8] = { -3 6 2 0 8 7 10 4 }
(indexes: 0 1 2 3 4 5 6 7)

K=3: -3 0 2

//idea 1

↳ Sort the array and return the first K elements.

T.C: O(NlogN + K)

//idea 2

↳ Add all the elements to min PQ and get the first K elements.

T.C: O(NlogN + KlogN)

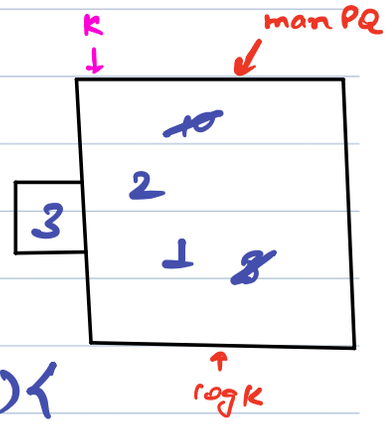
AlgoPrep

//idea 3

K = 3

	0	1	2	3	4	5	6	7	8	9
arr[10] = {	8	3	10	4	11	2	7	6	14	1 }

↳ 1 2 3



//Pseudo code

```
void Kthsmallest (int arr[N], int K){
    maxHeap < Integer> mh;

    for (int i=0; i<K; i++){
        mh.add (arr[i]);
    }

    for (int i=K; i<N; i++){
        if (arr[i] < mh.peek()){
            mh.remove();
            mh.add (arr[i]);
        }
    }

    while (mh.size() >0){
        s.o.p (mh.remove());
    }
}
```

T.C: O(N log K)

S.C: O(K)

AlgoPrep

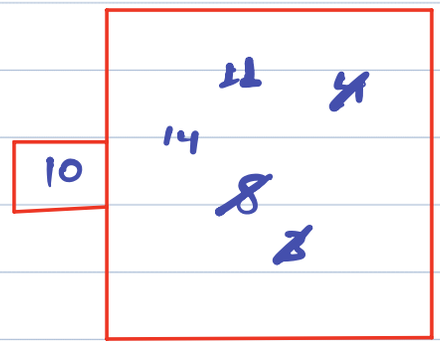
Q) K largest elements

	0	1	2	3	4	5	6	7	8	9
arr[10]:	8	3	10	4	11	2	7	6	14	1

K = 3

↳ 10 11 14

min PQ



AlgoPrep

//median

↳ middle element of sorted number.

arr[3] : { 2 5 3 }

↳ { 2 3 5 } → 3

arr[5] : { 4 3 6 8 5 }

↳ { 3 4 5 6 8 } → 5

arr[6] : { 4 3 9 5 12 2 }

↳ { 2 3 4 5 9 12 } → $\frac{4+5}{2}$ = 4.5

arr[4] : { 4 6 10 14 }

↳ $\frac{6+10}{2}$ = 8

Break till 9:18 pm

AlgoPrep

LeetCode 295

Q) Print median after each insertion.

arr[5]: 9 6 3 10 4

9 7.5 6 7.5 6

//idea 1

↳ After every insertion, sort the array and return the middle one.

T.C: N*NlogN ≃ O(N²logN)

//idea 2

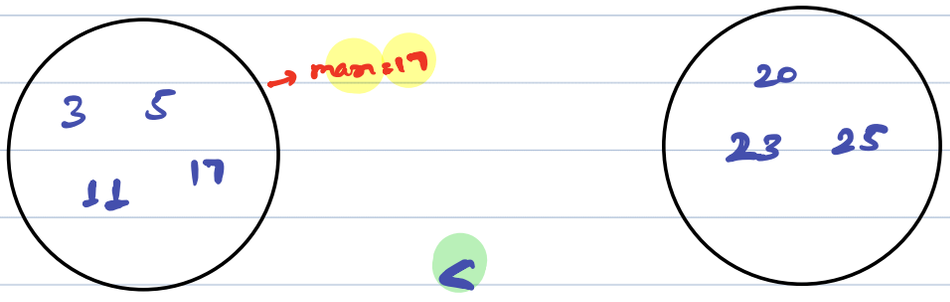
3 5 11 23 20 25 17 7

* if element count is even:

$$\text{median} = \frac{\text{max of left bucket} + \text{min of right bucket}}{2}$$

* if element count is odd:

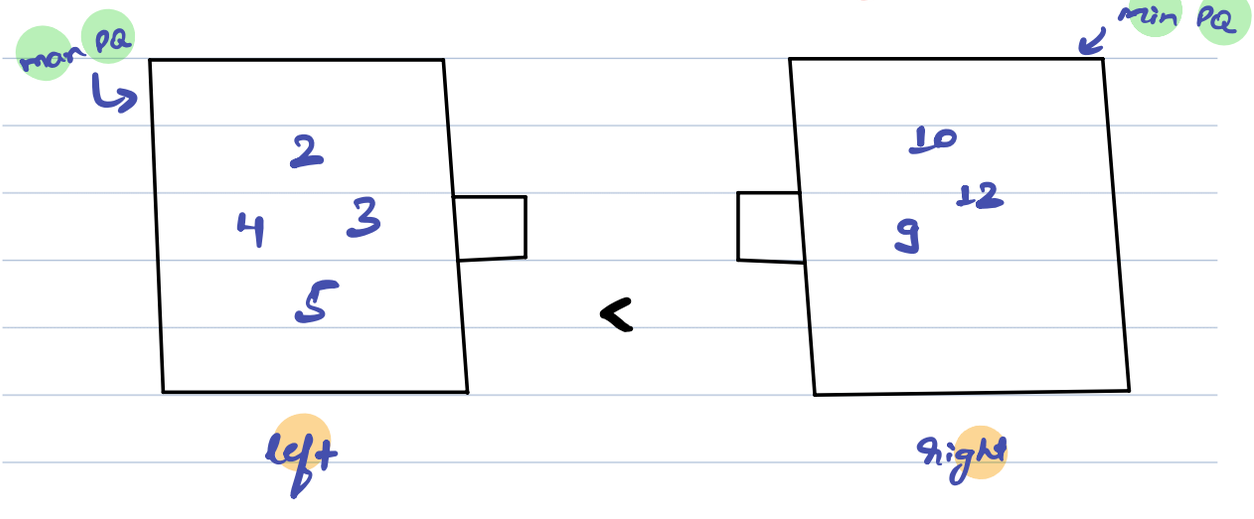
3 5 11 23 20 25 17



median = max of left

→ 4 9 5 3 12 2 10

4 6.5 5 4.5 5 4.5 5



if (left.size() == right.size()) {

↳ ultimately new number should go to left PQ but to maintain inequality you have to pass it via right PQ.

}

if (left.size() != right.size()){

↳ ultimately new number should go to right PQ but to maintain inequality you have to pass it via left PQ.

}

AlgoPrep

//Pseudo code

```
Class MedianFinder {
    maxHeap<Integer> left;
    minHeap<Integer> right;

    Public MedianFinder() {

    }
    Public void addNum(int num) {
        if (left.size() == right.size()) {
            right.add(num);
            left.add(right.remove());
        }
        else {
            left.add(num);
            right.add(left.remove());
        }
    }

    Public double findMedian() {
        if (left.size() == right.size()) {
            return (left.Peek() + right.peek())/2.0;
        }
        else {
            return left.peek()*1.0;
        }
    }
}
```

T.C: O(N log N)

S.C: O(N)

3 log N

O(1)